Dvodimenzionalne vložitve podatkov

Nadomestni naslov tega poglavja bi lahko bil "podatkovne karte", saj v njem opišemo tehnike, ki podatke preslikajo na dvodimenzionalno ravnino, kjer so posamezni primeri točke in kjer lahko iz oddaljenosti med točkami sklepamo o odnosih med primeri oziroma o razdaljah v njihovem originalnem prostoru. A bi s tem opisom morali med take pristope šteti že metodo glavnih komponent. Tako pa smo se zatekli k drugačnemu naslovu, in tu govorimo o vložitvah podatkov v nižjedimenzionalne prostore, oziroma kar v dvodimenzionalne ravnine, saj nas prvenstveno zanimajo vizualizacije podatkovnega prostora.

Potem analiza glavnih komponent ni vložitev podatkov? Ne, ni. Metoda glavnih komponent je projekcijska metoda, ki oblikuje matematični predpis, recimo U (matrika, sestavljena iz smernih vektorjev glavnih komponent u_1 in u_2), ki podatke X projicira v dve dimenziji, torej $Z = XU$. Projekcija, v splošnem, ni vložitev. Za vložitve, ki jih bomo opisali v tem poglavju, bomo namreč predpisali kriterijsko funkcijo, kateri bodo morale točke zadostiti, in pri tem ne bomo iskali matematične preslikave $f(x)$, ampak le optimalno postavitev točk $y_1, \ldots, y_n \in \mathbb{R}^2$. ki minimizira izbrano kriterijsko funkcijo. Takšna postavitev je določena neposredno na ravni medsebojnih odnosov med primeri, na primer njihovih razdalj ali podobnosti, in praviloma ne omogoča enostavne preslikave novih podatkov v isti prostor. V tem smislu vložitve ne gradijo novega koordinatnega sistema, temveč predvsem vizualno predstavitev strukture podatkov.

Oznaka y_i tu ne predstavlja razreda (kot pri napovednih modelih), temveč koordinate točke v vložitvi primera x_i oziroma položaj tega primera v dvodimenzionalnem prostoru.

Ogledali si bomo tri tehnike vložitev: prvo, ki je pomembna tudi z zgodovinskega vidika razvoja tega področja, drugo, ki je v zadnjem času pridobila veliko priljubljenost (tako metoda kot njen avtor), in tretjo, ki izhaja iz tehnik vizualizacije omrežij. Videli bomo, da kljub popolnoma različnim kriterijskim funkcijam te metode na manjših podatkovnih množicah pogosto vodijo do zelo podobnih rezultatov. Pri velikih podatkih, z mnogo primeri in iz visokorazsežnih prostorov, pa se razlike med metodami lahko izrazito povečajo.

Skupen vsem trem opisanim metodam je osnovni pristop vložitve primerov v dve dimenziji. Uporabili bomo kriterijsko funkcijo, ki

izhaja iz razdalj med primeri v izvornem prostoru in jih na ustrezen način poveže s položaji točk v vložitvenem prostoru. Začetno vložitev dobimo tako, da točke (primere) v ravnini razporedimo naključno, nato pa njihove položaje postopoma popravljamo v smeri gradienta kriterijske funkcije glede na koordinate teh točk. Tokrat torej model ne predstavljajo uteži ali parametri preslikave, temveč kar same koordinate točk v vložitvi, zato je število parametrov enako $2n$, kjer je n število primerov.

Podatki

Začnimo kar s podatki. Vse metode, ki jih bomo obravnavali v tem poglavju, na vhodu prejmejo razdalje med primeri. Tokrat primerov torej ne bomo opisovali eksplicitno z atributi, kot smo bili vajeni v prejšnjih poglavjih, temveč zgolj z razdaljami med njimi. Te razdalje bi običajno zapisali v (simetrični) matriki, lahko pa jih predstavimo tudi s spodnjim slovarjem:

Lahko bi seveda tudi pričeli s primeri v atributnem zapisu, a bi potem morali uporabiti neko mero razdalje. Plus, očitno lahko, kot bomo počeli v tem poglavju, podatke predstavimi samo z razdaljami med njimi. So lahko taki podatki vhod v PCA?

```
distances = {
    ("Novo Mesto", "Maribor"): 170,
    ("Novo Mesto", "Celje"): 83,
    ("Novo Mesto", "Koper"): 169,
    ("Novo Mesto", "Kranj"): 99,
    ("Novo Mesto", "Ljubljana"): 72,
    ("Novo Mesto", "Postojna"): 116,

    ("Maribor", "Celje"): 55,
    ("Maribor", "Koper"): 232,
    ("Maribor", "Kranj"): 156,
    ("Maribor", "Ljubljana"): 128,
    ("Maribor", "Postojna"): 178,

    ("Celje", "Koper"): 183,
    ("Celje", "Kranj"): 105,
    ("Celje", "Ljubljana"): 77,
    ("Celje", "Postojna"): 130,

    ("Koper", "Kranj"): 128,
    ("Koper", "Ljubljana"): 107,
    ("Koper", "Postojna"): 58,

    ("Kranj", "Ljubljana"): 30,
    ("Kranj", "Postojna"): 77,

    ("Ljubljana", "Postojna"): 53,
}
```

Večrazsežno lestvičenje

Zapišimo razdaljo med primeroma i in j z d_{ij}. V večrazsežnem lestvičenju želimo primere urediti v dvodimenzionalnem prostoru tako, da vsak primer i opišemo s koordinatama $y_i = (y_{i1}, y_{i2}) \in \mathbb{R}^2$. V tem ciljnem prostoru naj bo razdalja med primeroma evklidska,

$$\|y_i - y_j\| = \sqrt{(y_{i1} - y_{j1})^2 + (y_{i2} - y_{j2})^2},$$

torej takšna, kot bi jo dobili pri običajnem razsevnem diagramu.

Večrazsežno lestvičenje (angl. *multidimensional scaling*, MDS) so razvili Torgerson (1952), Shepard (1962) in Kruskal (1964) v okviru psihometrije, področja, ki se ukvarja z merjenjem psiholoških lastnosti (npr. zaznav, podobnosti med dražljaji in preferenc) z uporabo statističnih metod.

Cilj večrazsežnega lestvičenja je čim bolje ohraniti dane razdalje, torej naj bodo razdalje $\|y_i - y_j\|$ v vložitvenem prostoru čim bližje izhodiščnim razdaljam d_{ij}. Skladno s tem večrazsežno lestvičenje minimizira kriterijsko funkcijo

$$J(Y) = \sum_{i<j} \big(\|y_i - y_j\| - d_{ij}\big)^2,$$

kjer $Y = \{y_1, \ldots, y_n\}$ predstavlja iskano postavitev točk v ravnini.

Izraz *lestvičenje* (angl. *scaling*) pomeni razporejanje objektov na lestvico tako, da razdalje med njimi odražajo njihove medsebojne razlike ali podobnosti. Pridevnik *večrazsežno* (angl. *multidimensional*) poudarja, da te lestvice ne tvorimo le v eni dimenziji, temveč v večrazsežnem prostoru, najpogosteje v dveh ali treh dimenzijah.

Evklidska razdalja v vložitvenem prostoru je torej v celoti odvisna od položajev točk, ki predstavljajo primere; ti položaji pa so tudi parametri našega modela. Te bomo iskali z gradientnim sestopom, pri čemer bomo uporabili strojno odvajanje. Knjižnico za strojno odvajanje in učenje z gradientnim sestopom smo že zgradili v prejšnjih poglavjih, zato tu podajamo le kodo razreda za predstavitev modela večrazsežnega lestvičenja.

```
class MDS:
    def __init__(self, items):
        random.seed(100)
        self.pos = {i: [Value(random.uniform(-1, 1)) for _ in range(2)]
                    for i in sorted(items)}

    def parameters(self):
        return [p for pos in self.pos.values() for p in pos]

    def d(self, a, b):
        return sum([(ai - bi) ** 2 for ai, bi in \
                    zip(self.pos[a], self.pos[b])]) ** 0.5

    def loss(self, xs, ys):
        n = len(xs)
        return sum((self.d(*pair) - y) ** 2 \
                   for pair, y in zip(xs, ys)) / Value(n)
```

Razred `MDS` zgradi računski graf kriterijske funkcije za večrazsežno lestvičenje za dane vhodne podatke (razdalje). Vsakemu primeru priredimo naključno začetno pozicijo v ravnini (`self.pos`), te koordinate pa predstavljajo parametre modela. Metoda `d` izračuna evklid-

sko razdaljo med dvema primeroma v trenutni vložitvi, `loss` pa meri odstopanje med temi razdaljami in podanimi razdaljami d_{ij}.

Z gradientnim sestopom nato popravljamo položaje točk tako, da se ta napaka čim bolj zmanjša:

```
items = sorted(list(set(i for pair in distances.keys() for i in pair)))
pairs = list(distances.keys())
targets = [distances[p] for p in pairs]

model = MDS(items)
model = train(model, pairs, targets, n_epochs=1000,
              learning_rate=0.1, report_every=100)
```

Mest je v naši učni množici malo, optimizacija pa konvergira hitro. Namesto izpisa pozicij posameznih mest je tu seveda bolj na mestu izris podatkovne karte:

```
plt.figure(figsize=(4, 4))
for city in sorted(items):
    x, y = model(city)
    plt.scatter(x.data, y.data, color="k", s=10)
    plt.annotate(
        city,
        (x.data, y.data),
        xytext=(0, 4),
        textcoords="offset points",
        ha="center",
        va="bottom",
    )
plt.savefig("mds-cities.pdf")
```

Karto mest prikazuje slika 11. Malce je zasukana in narobe obrnjena, a kdo pravi, da mora biti sever na vrhu in da gledamo od zgoraj. Relacije med mesti so sicer kar prave. Maribor, Celje, Ljubljana, Postojna in Koper so na glavni diagonali v pravem vrstnem redu, Kranj malce stran, Novo mesto na drugi strani. Vhodni podatki so vsebovali cestne razdalje med mesti, ki niso enake zračnim; evklidska razdalja v prostoru vložitve pa ustreza zračni. Najbrž je to glavni izvor za kakšne manjše napake, a v splošnem nam je rekonstrukcija pozicije slovenskih mest iz cestnih razdalj dobro uspela.

Opozorilo sicer: MDS-vložitev z enako dobro vrednostjo kriterijske funkcije $J(Y)$ je nešteto. Pri nespremenjeni vrednosti kriterijske funkcije vložitve lahko poljubno rotiramo in zrcalimo. Vse te transformacije ohranjajo razdalje in pravzaprav samo te nastopajo v kriterijski funkciji. Dobljene osi zato nimajo pomena in jih kot takih ne moremo povezati s kakšno razlago. Drugače kot pri PCA torej. A spomnimo: pri PCA izhajamo iz atributno predstavljenih primerov, tu pa, pri MDS, iz razdalj.

Našo karto slovenskih mest bi lahko za lažjo primerjavo s geografskimi kartami, na katere smo navajeni, zasukali za 180 stopinj.

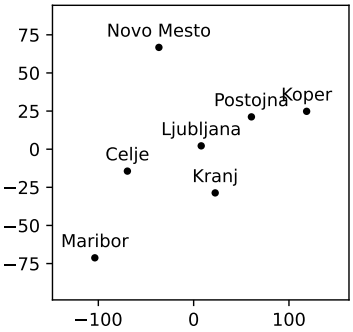

Figure 9: Dvodimenzionalna vložitev z večrazrednim lestvičenjem.

Vložitve sosedov

Večrazredno lestvičenje ima skriti problem: kriterijska funkcija kvadrira razliko razdalj iz vhodnih podatkov in vložitvenega prostora. Učenje modela z gradientnim sestopom se bo zato bolj "potrudilo" pri zmanjševanju večjih odstopanj. Če odstopanja merimo v deležih, na primer v odstotkih, in če na primer na vložitvi razdalje odstopajo od vhodnih za 10%, bo odstopanje med Ljubljano in Kranjem imelo dosti manjši vpliv na učenje kot odstopanje med Koprom in Mariborom. MDS torej teži k ohranjanju predvsem tistih večjih razdalj. Vpliv krajših razdalj med sosednjimi primeri je veliko manjši.

Ohranjanje večjih razdalj je seveda zaželeno, če želimo pridobiti podatkovne karte s primernimi globalnimi ureditvami. A mnogokrat nas pri podatkovni analitiki zanimajo predvsem soseščine in na podatkovnih kartah morda iščemo samo skupine podatkov ter jih skušamo interpretirati, pri čemer nas globalni odnosi oziroma skupine, ki so oddaljene druga od druge, ne zanimajo. Tako bi nas zanimale vizualizacije, kjer so izražene skupine primerov, ki so si med sabo podobni, in nas sama interpretacija razdalje med skupinami, ki so si med seboj oddaljene, ne zanima. Taka vizualizacija je na primer prikazana na sliki **??** na desni strani in je očitno zelo različna od vizualizacije istih podatkov, kot jo pridela MDS.

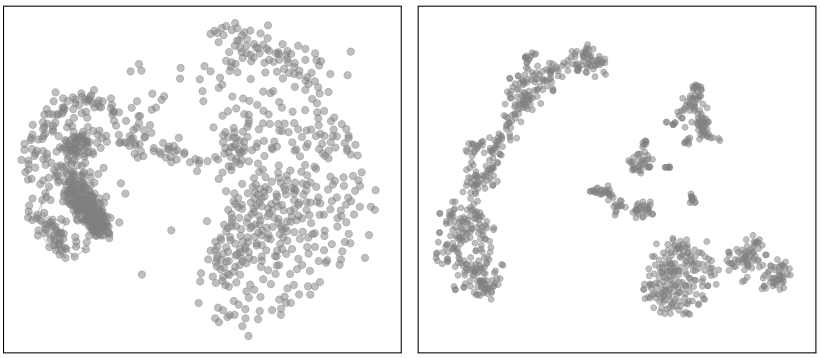

Figure 10: Primerjava vložitev MDS (levo) in t-SNE (desno) na podatki o izražanju genov v celicah kostnega mozga (vzorec tisoč celic in izrazi tisoč genov). Skupine točk naj bi pripadale posameznim celičnim tipom.

Zanima nas torej ohranjanje soseščine. Za to potrebujemo mero, ki pove, kako blizu so si primeri v izvornem prostoru, in mero, ki pove, kako blizu so si v prostoru vložitev. Ti meri morata zanemariti primere, ki so si daleč, in se osredotočiti le na bližnje primere. Tu lahko uporabimo trik: namesto razdalje bomo ocenili verjetnost, da sta si dva primera blizu, v izvornem prostoru in prostoru vložitev, kot kriterijsko funkcijo pa ocenili podobnost med tema vektorjema verjetnosti, kjer z vektorjema mislimo seznama verjetnosti bližine vseh parov primerov.

Zanima nas torej ohranjanje soseščine. Za to potrebujemo mero, ki pove, kako blizu so si primeri v izvornem prostoru, in mero, ki

pove, kako blizu so si v prostoru vložitev. Ti meri morata zanemariti primere, ki so si daleč, in se osredotočiti le na bližnje primere. Tu lahko uporabimo trik: namesto razdalje bomo ocenili verjetnost, da sta si dva primera blizu, v izvornem prostoru in prostoru vložitev, kot kriterijsko funkcijo pa ocenili podobnost med tema vektorjema verjetnosti, kjer z vektorjema mislimo seznama verjetnosti bližine vseh parov primerov.

Lotimo se tega bolj formalno. Kot pri MDS naj bo x_i primer v izvornem prostoru in y_i njegova predstavitev v vložitvi. Za vsak par primerov (i, j) določimo podobnost v izvornem prostoru kot pogojno verjetnost

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}.$$

Ta verjetnost pove, kako verjetno bi izbrali primer j kot soseda primera i. Pri tem parameter σ_i določa "širino" okolice okoli x_i. Ker pričakujemo, da je funckcija $p_{j|i}$ simetrična, torej, da, velja $p_{j|i} = p_{i|j}$, združimo obe verjetnosti v njuno povprečje in normaliziramo:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n},$$

kjer je n število vseh primerov.

V prostoru vložitev podobno določimo verjetnosti bližine, vendar uporabimo porazdelitev z debelejšimi repi (Studentovo t-porazdelitev z eno stopnjo prostosti):

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}.$$

Ta izbira omogoča, da so oddaljeni primeri v vložitvi še vedno dovolj "ločeni" in se ne zlepijo skupaj.

Cilj tega pristopa je poiskati takšne vložitve y_i, da so porazdelitve p_{ij} in q_{ij} čim bolj podobne. To dosežemo z minimizacijo Kullback-Leiblerjeve divergence:

$$\mathcal{L}(\dagger) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

Ta kriterijska funkcija kaznuje predvsem primere, kjer sta si točki v izvornem prostoru blizu (velik p_{ij}), v vložitvi pa daleč narazen (majhen q_{ij}), zato spodbuja ohranjanje lokalne strukture podatkov.

Kriterijsko funkcijo, kot je določena zgoraj, uporablja metoda t-SNE (angl. *t-distributed Stochastic Neighbor Embedding*). Ta pristop je nadgradil starejšo metode SNE, ki je prav tako temeljila na primerjavi verjetnostnih porazdelitev bližin med pari primerov v izvornem

Metodo SNE sta predlagala Geoffrey E. Hinton in Sam T. Roweis (2002), t-SNE pa Hinton in Laurens van der Maaten (2008). Hinton je tudi eden od očetov sodobne umetne inteligence je leta 2024 za svoje raziskovalno delo prejel Nobelovo nagrado.

prostoru in prostoru vložitev. A je imel SNE nekaj pomembnih pomanjkljivosti. Prvič, uporabljal je nesimetrične verjetnosti $p_{j|i}$ in $q_{j|i}$, kar je otežilo optimizacijo in interpretacijo. Drugič, zaradi uporabe Gaussove porazdelitve tudi v prostoru vložitev je trpel za t. i. problemom "zgoščanja" (angl. *crowding problem*), kjer se večje razdalje težko ustrezno predstavijo v nizkodimenzionalnem prostoru, zato se točke pogosto preveč nagnetejo skupaj.

Metoda t-SNE odpravi te težave. Uvede simetrične verjetnosti p_{ij} in q_{ij}, kar poenostavi kriterijsko funkcijo in v prostoru vložitev uporabi Studentovo t-porazdelitev z enim prostostnim parametrom, ki ima daljši rep kot Gaussova porazdelitev. To omogoča, da so si oddaljeni primeri v vložitvi bolj naravno razmaknjeni, s čimer se zmanjša problem zgoščanja in izboljša prikaz lokalne strukture podatkov.

Prestavimo sedaj zgornje matematične izraze v kodo. Znova je vse, kar potrebujemo, razred, ki zgradi računski graf za kriterijsko funkcijo. Neverjetno, kaj vse lahko pričnemo s pristopom strojnega odvajanje, kajne?

```
class tSNE:
    def __init__(self, items, sigma=40.0, eps=1e-12):
        random.seed(100)
        self.pos = {
            i: [Value(random.uniform(-1, 1)) for _ in range(2)]
            for i in sorted(items)
        }
        self.sigma = sigma  # constant bandwidth of the Gaussian
        self.eps = eps  # parameter for numerical stability

    def parameters(self):
        return [p for pos in self.pos.values() for p in pos]

    def p_distribution(self, pairs, distances):
        unnorm = []
        for d in distances:
            pij = math.exp(-(d ** 2) / (2 * self.sigma ** 2))
            unnorm.append(pij)

        z = sum(unnorm) + self.eps
        return [p / z for p in unnorm]

    def sqdist(self, a, b):
        return sum((ai - bi) ** 2 \
                   for ai, bi in zip(self.pos[a], self.pos[b]))

    def q_distribution(self, pairs):
        numerators = []
        for a, b in pairs:
```

```
            numerators.append((Value(1.0) + self.sqdist(a, b)) ** -1)

        z = sum(numerators)
        return [q / z for q in numerators]

    def loss(self, pairs, distances):
        p = self.p_distribution(pairs, distances)
        q = self.q_distribution(pairs)

        loss = Value(0.0)
        for pij, qij in zip(p, q):
            loss = loss + Value(pij) * \
              (math.log(pij + self.eps) - (qij + self.eps).log())

        return loss
```

Razred 'tSNE' v konstruktorju '__init__' najprej za vsak primer naključno inicializira dvodimenzionalno vložitev 'self.pos'. Metoda 'p_distribution' iz podanih razdalj v izvornem prostoru izračuna porazdelitev podobnosti p_{ij} tako, da vsako razdaljo pretvori v utež z Gaussovo funkcijo in nato vse uteži normalizira v verjetnosti. Za prostor vložitev metoda 'sqdist' izračuna kvadrat evklidske razdalje, 'q_distribution' pa to uporabi tako da iz trenutnih položajev točk izračuna porazdelitev q_{ij}. Pri tem uporabi jedro oblike $(1 + |y_i - y_j|^2)^{-1}$. Kriterijsko funkcijo smo implementirali v metodi 'loss', ki najprej določi porazdelitvi 'p' in 'q', potem pa za izračun Kullback-Leiblerjeve divergence za vse pare sešteje $p_{ij} \log(p_{ij}/q_{ij})$.

Tu velja opozoriti, da smo zgoraj predstavili poenostavljeno različico metode t-SNE. Avtorji izvorne rešitve so namreč predlagali, da se širina Gaussove porazdelitve σ_i ne določi vnaprej in za vse točke enako, ampak ta parameter, ki določa mero razpršenosti, za vsak primer posebej prilagodimo tako, ki nadzoruje velikost lokalne soseščine (angl. *perpleksnost*). Prav tako se pogosto uporablja dodatne izboljšave optimizacije, kot so zgodnje pretiravanje (angl. *early exaggeration*), kjer v začetnih iteracijah umetno povečamo vrednosti verjetnosti p_{ij} (tipično jih pomnožimo z neko konstanto). S tem okrepimo privlačne sile med bližnjimi točkami, zato se skupine podatkov najprej jasno oblikujejo in ločijo med seboj, še preden začne optimizacija natančneje uravnavati njihove medsebojne razdalje. Po nekaj korakih ta učinek izklopimo, kar omogoči stabilnejše končno razporejanje točk.

Uporaba zgornje implementacije od tu dalje ni prav nič drugačna kot za MDS:

```
model = tSNE(items, sigma=40.0)
model = train(model, pairs, targets,
              n_epochs=3000, learning_rate=0.05, report_every=100)
```

Tudi tu so na vhodu razdalje, na izhodu pa vložitve primerov. Te znova lahko prikažemo na razsevnem diagramu (slika 11). Hm. Vložitev ni prav dosti drugačna od te pri MDS. Kar je sicer zanimivo, saj gre za metodi, ki imata popolnoma drugačno kriterijsko funkcijo. Razlike med MDS in t-SNE res nastopijo, ko je podatkov več, in ko ti prihajajo iz mnogodimenzionalnih prostorov. V visokih dimenzijah se evklidske razdalje "zgoščajo", kar pomeni, da postanejo razlike med bližnjimi in daljnimi primeri manj izrazite, zato globalna struktura izgubi pomen, lokalne soseščine pa postanejo ključne. Pri večjem številu podatkov pa je tudi lokalna struktura bogatejša in bolj zanesljiva, zato jo lahko t-SNE učinkoviteje izpostavi, medtem ko MDS še vedno poskuša ohraniti globalne razdalje in s tem "zgladi" te lokalne vzorce. Primer take, zelo različne vložitve kaže slika 10, kjer je pri MDS težko med sabo ločiti skupine, ki so jasno razvidne v viuzaliacije vložitev t-SNE.

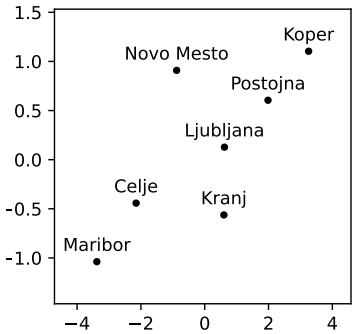

Figure 11: Dvodimenzionalna vložitev z večrazrednim lestvičenjem.

Vložitve s silami

Oglejmo si še tretji pristop. Podamo ga zato, ker je znova skoraj popolnoma drugačen od prejšnjih dveh, in ker je silno znan na področju vizualizacije omrežij. Pristop predpostavi, da so primeri točke, med katerimi delujejo sile. Pari primerov, za katere poznamo razdalje, Tu smo odprli še en nov koncept: do sedaj so podatki, pač tako je naneslo, vsebovali razdalje med vsemi primeri. A teh, za delovanje naših metod, sploh ne potrebujemo. Nekoliko več o tem v nadaljevanju. povežemo z "vzmetmi", ki skušajo ohranjati predpisano dolžino, hkrati pa med vsemi pari deluje še odbojna sila, ki preprečuje, da bi se točke preveč zgostile. Položaj točk v ravnini nato dobimo z iskanjem ravnovesja sil, ki ga poiščemo z minimizacijo ustrezne energijske funkcije. Čeprav je tak pristop bil razvit na področju vizualizacije omrežij, kjer želimo pregledno razporediti vozlišča grafa, ga enako uspešno uporabimo tudi za vložitve podatkov.

Kot prej, naj bo $y_i \in \mathbb{R}^2$ znova položaj primera i v ravnini. Za pare primerov (i, j), za katere poznamo razdaljo d_{ij}, uvedemo "vzmetno" energijo

$$E_{\text{vzmeti}} = \frac{1}{|E|} \sum_{(i,j)\in E} \left(\|y_i - y_j\| - d_{ij}\right)^2,$$

kjer E označuje množico parov z znanimi razdaljami. Ta člen sili povezane pare, da v vložitvi ohranjajo predpisane razdalje.

Med vsemi pari primerov hkrati deluje še odbojna sila, ki jo

opišemo z energijo

$$E_{\text{odboj}} = \frac{1}{\binom{n}{2}} \sum_{i<j} \frac{C}{\|y_i - y_j\|},$$

kjer konstanta C določa jakost odboja. Ta člen preprečuje, da bi se točke preveč približale.

Ker želimo preprečiti, da bi celotna konfiguracija "odplavala" stran od izhodišča, dodamo še šibek centrirni člen

$$E_{\text{center}} = \frac{\lambda}{n} \sum_{i=1}^{n} \|y_i\|^2,$$

kjer λ določa njegovo jakost.

Skupno energijo zapišemo kot vsoto vseh treh členov

$$E = E_{\text{vzmeti}} + E_{\text{odboj}} + E_{\text{center}},$$

in poiščemo takšne položaje točk y_i, ki to energijo minimizirajo.

Zgornja kriterijska funkcija je na prvi pogled še najbolj podobna tej, ki jo uporablja MDS. A podobnost je zgolj navidezna in velja le, ko so razdalje podane za vse pare. Pri MDS en sam člen enakovredno obravnava vse pare primerov in skuša globalno ohraniti vse razdalje. Pri silno vodenih vložitvah pa se kriterijska funkcija razcepi na več delov: vzmeti delujejo le med izbranimi pari (npr. povezanimi v omrežju) in skrbijo za lokalno strukturo, odboj pa deluje med vsemi pari in preprečuje zgoščanje točk. S tem dobimo ravnovesje med lokalnim ohranjanjem razdalj in globalnim razmikom točk, kar vodi do bistveno drugačnih vložitev kot pri MDS.

Tokrat je implementacijska koda, pač zaradi treh členov kriterijske funkcije, malce daljša:

```
class ForceDirectedLayout:
    def __init__(self, items, repulsion_strength=5000.0, center_strength=0.001):
        random.seed(100)
        self.items = sorted(items)
        self.repulsion_strength = repulsion_strength
        self.center_strength = center_strength

        self.pos = {
            i: [Value(random.uniform(-1, 1)) for _ in range(2)]
            for i in self.items
        }

    def parameters(self):
        return [p for pos in self.pos.values() for p in pos]

    def d(self, a, b):
        dx = self.pos[a][0] - self.pos[b][0]
```

```
        dy = self.pos[a][1] - self.pos[b][1]
        return (dx ** 2 + dy ** 2 + Value(1e-6)) ** 0.5

    def loss(self, edges, target_lengths):
        # springs between connected pairs
        spring_loss = Value(0.0)
        n_edges = len(edges)

        for (a, b), target in zip(edges, target_lengths):
            dist = self.d(a, b)
            spring_loss += (dist - Value(target)) ** 2

        spring_loss = spring_loss / Value(n_edges)

        # repulsion between all pairs
        repulsion_loss = Value(0.0)
        n = len(self.items)
        repulsion_count = 0

        for i in range(n):
            for j in range(i + 1, n):
                a = self.items[i]
                b = self.items[j]
                dist = self.d(a, b)
                repulsion_loss += Value(self.repulsion_strength) / dist
                repulsion_count += 1

        repulsion_loss = repulsion_loss / Value(repulsion_count)

        # centering loss
        center_loss = Value(0.0)
        for city in self.items:
            x, y = self.pos[city]
            center_loss += x * x + y * y

        center_loss = Value(self.center_strength) * center_loss / Value(n)

        return spring_loss + repulsion_loss + center_loss
```

Na prav enak način kot pri MDS in t-SNE pridobimo tudi vložitve s silami. Tudi tu je konvergenca hitra, dobljena vizualiza Vložitve tudi tokrat niso prav